

LABORATORIO DE PRUEBAS AUTOMATIZADAS CON CYPRESS

Juan Jose Santacruz Ferraro

1. INTRODUCCIÓN

En este laboratorio usamos Cypress para hacer pruebas end-to-end a una aplicación desarrollada en React. El componente principal fue un contador con funcionalidades como incrementar, decrementar, reiniciar y trabajar con valores personalizados. Cypress nos permitió automatizar la validación del comportamiento de este componente directamente en el navegador.

2. OBJETIVOS

- Configurar e implementar pruebas end-to-end utilizando Cypress.
- Validar la funcionalidad completa del contador desde la vista del usuario.
- Automatizar pruebas que simulan interacciones reales como clics, ingreso de datos y validaciones de texto.

3. DESARROLLO

3.1 Instalación

Se instaló Cypress con el comando:

```
npm install cypress --save-dev
```

Luego se ejecutó Cypress por primera vez con:

```
npx cypress open
```

Welcome to Cypress!

[Review the differences between each testing type →](#)



E2E Testing

Build and test the entire experience of your application from end-to-end to ensure each flow matches your expectations.

Configured



Component Testing

Build and test your components from your design system in isolation in order to ensure each state matches your expectations.

Not Configured

Choose a browser

Choose your preferred browser for E2E testing.



Chrome
v136



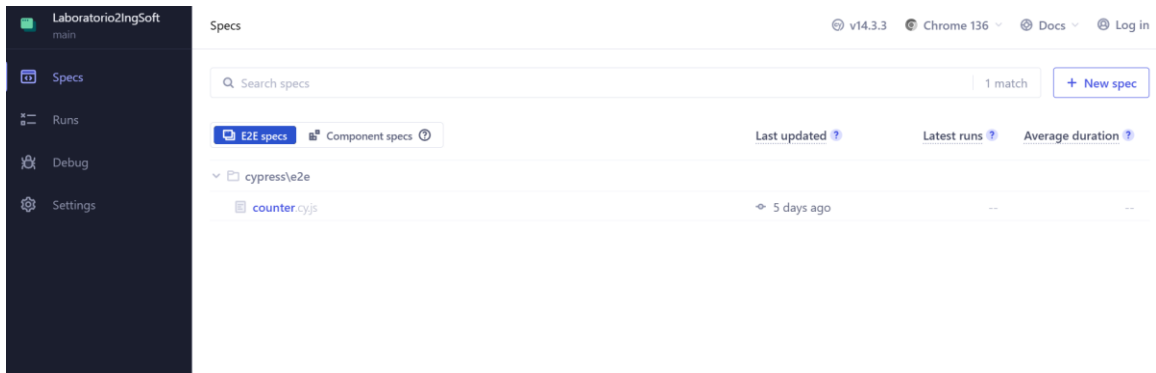
Edge
v136



Electron
v130

Start E2E Testing in Chrome

[← Switch testing type](#)



3.2 Configuración

Al iniciar, Cypress generó la estructura de carpetas:

- cypress/e2e
- cypress/support
- cypress/fixtures

También se configuró el archivo `cypress.config.js` con el entorno básico para E2E testing.

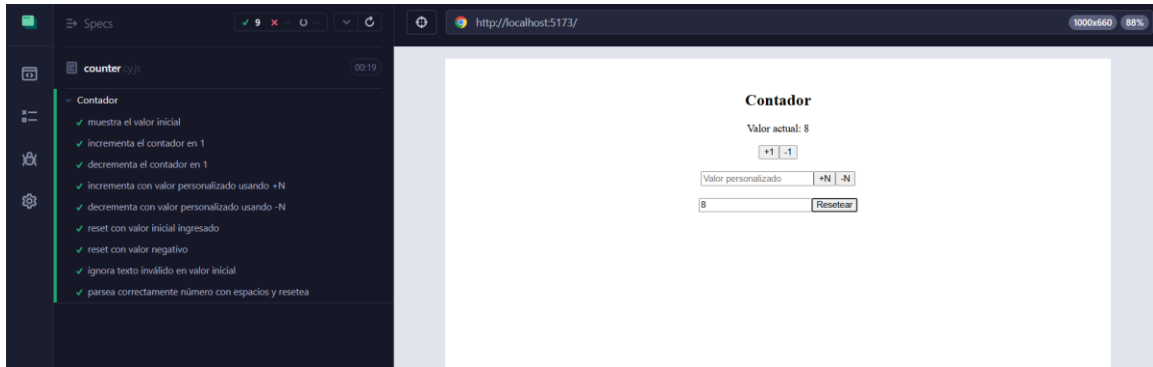
3.3 Pruebas creadas

Se creó un archivo llamado `counter.cy.js` dentro de `cypress/e2e/` con las siguientes pruebas:

```
counter.cy.js X
cypress > e2e > counter.cy.js > ...
1 describe('Contador', () => {
2   beforeEach(() => {
3     cy.visit('http://localhost:5173');
4     cy.wait(600);
5   });
6
7   it('muestra el valor inicial', () => {
8     cy.contains(/valor actual/i).should('exist');
9     cy.wait(600);
10  });
11
12  it('incrementa el contador en 1', () => {
13    cy.contains('+1').click();
14    cy.wait(600);
15    cy.contains('Valor actual: 1').should('exist');
16  });
17
18  it('decrementa el contador en 1', () => {
19    cy.contains('-1').click();
20    cy.wait(600);
21    cy.contains('Valor actual: -1').should('exist');
22  });
23
```

- Mostrar valor inicial del contador.
- Incrementar y decrementar por 1.
- Usar valores personalizados con +N y -N.
- Resetear el contador al valor ingresado.
- Manejar números negativos y entradas inválidas como texto o espacios.
- Validar que todo el flujo del contador funciona correctamente desde el navegador.

4. RESULTADOS



Todas las pruebas ejecutadas en el archivo counter.cy.js pasaron exitosamente. Cypress mostró visualmente cómo se comporta la app en el navegador y permitió validar que el contador responde a las acciones del usuario tal como se espera.

Esto demuestra que la aplicación cumple con los requisitos funcionales establecidos y responde correctamente incluso ante entradas inusuales como texto o espacios.

5. DIFICULTADES

- Al inicio hubo dudas sobre cómo ubicar los elementos con Cypress y cómo simular correctamente el ingreso de valores.
- En algunos casos fue necesario agregar tiempos de espera (cy.wait) para que los cambios en el DOM fueran detectados correctamente por Cypress.

6. CONCLUSIONES

- Cypress es una herramienta muy útil para validar visualmente y de forma automatizada el comportamiento de aplicaciones web.
- Me permite simular lo que haría un usuario real y verificar si el sistema responde correctamente.
- Automatizar pruebas de este tipo ayuda a detectar errores antes de que lleguen al usuario final y mejora la calidad del software.